

Champion Document — Maze Generation & Level Data

Algernon — Feature F2 (TL2) · MVP Core

Name: Nick Howard Mark: ____ / 30

1. Introduction

My feature for Algernon is Maze Generation & Level Data. It produces the playable maze and owns the grid that the other game systems query. At level start, the feature accepts a MazeConfig containing the seed and difficulty, selects an appropriate generation algorithm, builds the grid, validates that the maze is solvable, places the spawn and exit, registers typed anchor cells, and builds the Unity tilemap or wall geometry.

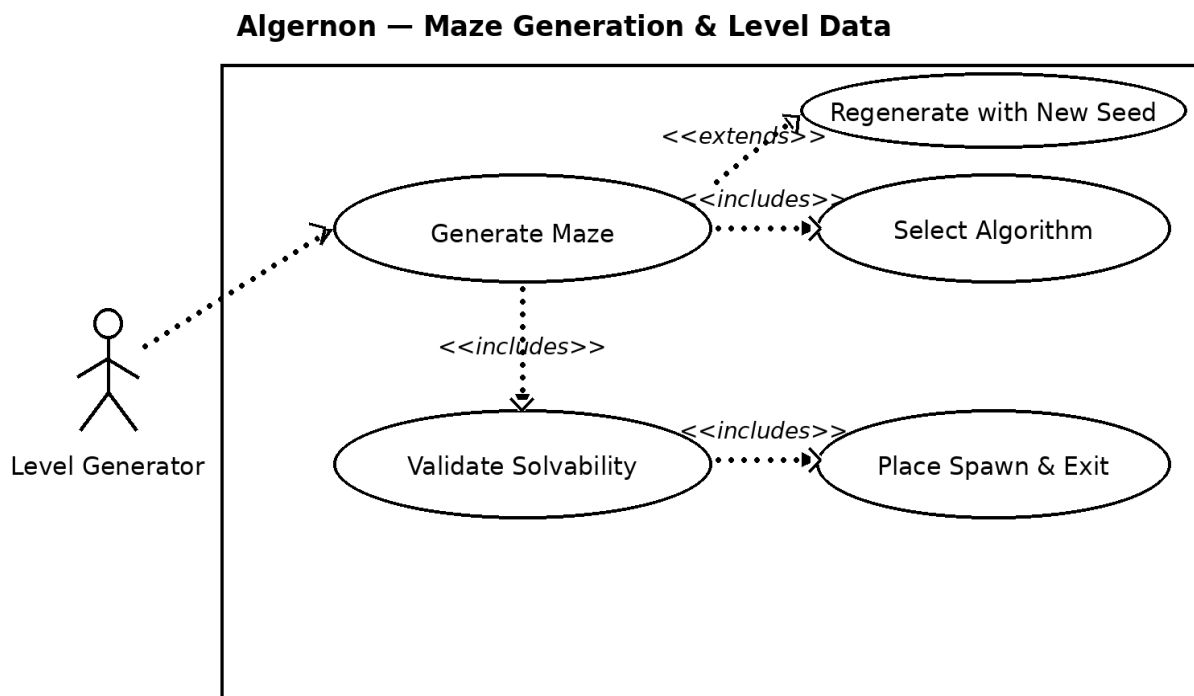
The feature also exposes a read-only query API for walkability, neighbouring cells, coordinate conversion, bounds, path existence, spawn/exit cells, and anchor reservation. Seeded generation is deterministic: the same seed and configuration must reproduce the same maze.

The supplied feature plan identifies this as the MVP core and gives an effort target of approximately 52 hours.

This feature does NOT own puzzle logic, hazard behavior, predator behavior, camera distortion, HUD/audio, or game-session win/loss arbitration. Those systems consume maze data through the public API. In particular, ReserveAnchors prevents F3/F4 content from occupying the same tile without Maze Generation needing to know what those other features are.

2. Use Case

Use Case Diagram — Scenario 1



Scenario 1 — Generate Maze for Level

Summary: When a level begins, the Game Session / Level Flow system supplies the maze configuration. The feature generates a deterministic grid, selects the algorithm, validates solvability, places spawn/exit cells, registers anchors, and returns MazeData for rendering and runtime queries.

Actors: Game Session / Level Flow system

Preconditions: MazeConfig exists; the configuration contains a seed and difficulty; the MazeGenerator façade is initialized.

1. The Game Session passes MazeConfig to MazeGenerator.Generate(MazeConfig config).
2. MazeConfig is validated and the difficulty band selects a MazeAlgorithm implementation.
3. The selected algorithm initializes the grid and carves the maze using a seeded Random instance.
4. Spawn and exit candidates are selected from walkable cells.
5. The generated grid is checked with PathExists(SpawnCell, ExitCell).
6. The feature stores the validated MazeData as Current and renders the maze geometry/tilemap.
7. Generate returns the MazeData result to the caller.

Exceptions:

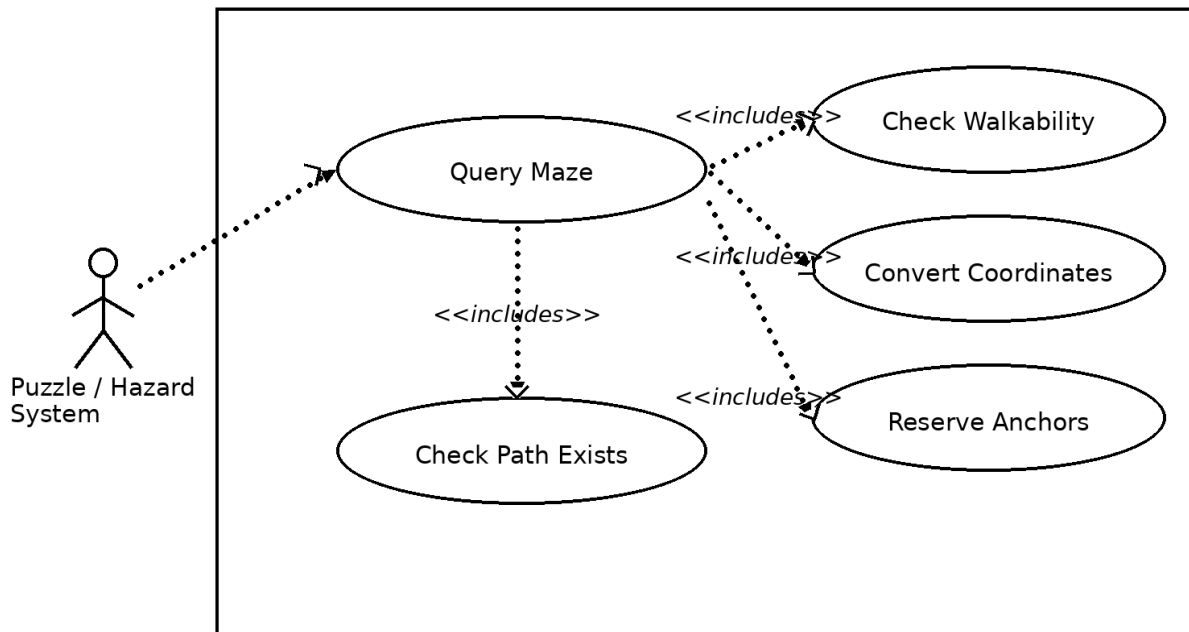
- Step 2a: The difficulty value is outside the supported range. The implementation rejects the configuration with a validation error rather than silently selecting an unrelated algorithm.
- Step 3a: The selected algorithm cannot complete the grid because of an invalid dimension/configuration. Generation terminates with a failure result and logs the seed and configuration for reproduction.
- Step 5a: Spawn and exit are not connected. The feature retries placement/generation using the deterministic configuration rules; after the configured retry limit, generation fails and reports the seed/configuration.
- Step 6a: Rendering cannot build the requested geometry. MazeData remains the authoritative generated result, while the renderer reports the rendering failure to the level-flow system.

Post conditions: Current contains a generated MazeData only after validation succeeds; SpawnCell and ExitCell are walkable and connected; rendered geometry matches the grid.

Priority: 1 (must have) ID: MG1

Use Case Diagram — Scenario 2

Algernon — Maze Generation & Level Data



Scenario 2 — Query Maze and Reserve Anchors

Summary: Puzzle and hazard systems request safe cells or maze information without needing to understand the maze's internal grid representation.

Actors: Puzzle / Hazard systems

Preconditions: Current contains a successfully generated MazeData.

8. A consuming feature requests walkability, bounds, coordinate conversion, path existence, or anchors.
9. The MazeGenerator façade forwards the request to Current MazeData / AnchorRegistry.
10. The feature checks the requested cell or computes the requested conversion/result.
11. For ReserveAnchors, the AnchorRegistry removes the selected cells from future reservation candidates.
12. The requested typed result is returned to the caller.

Exceptions:

- Step 1a: ReserveAnchors requests more anchors than are available. The method returns fewer than the requested count or a documented failure result; it never returns duplicate reservations.
- Step 3a: A requested cell is outside Bounds. IsWalkable returns false rather than accessing invalid grid memory.
- Step 4a: A cell has already been reserved. It is excluded from the new result so two consumers cannot receive the same anchor.

Post conditions: Queries have no side effects except ReserveAnchors, which records the cells it handed out.

Priority: 1 (must have) ID: MQ1

3. Interface Contract

The feature plan defines the public API below. Each operation has a concrete C# type so callers and tests can verify exact results. The feature plan does not name event types; the event signatures below are therefore proposed integration contracts, clearly separated from the source-defined API.

Operation	Typed signature	Testable contract
Generate	MazeData Generate(MazeConfig config)	Input: MazeConfig. Output: MazeData. Generates, validates, stores, and returns the current maze.
Current	MazeData Current { get; }	Output: MazeData. Read-only access to the successfully generated maze.
IsWalkable	bool IsWalkable(Vector2Int cell)	Input: Vector2Int. Output: bool; false for non-walkable/out-of-bounds cells.
SpawnCell	Vector2Int SpawnCell { get; }	Output: Vector2Int. Generated spawn cell.
ExitCell	Vector2Int ExitCell { get; }	Output: Vector2Int. Generated exit cell.
WorldBounds	Bounds WorldBounds { get; }	Output: UnityEngine.Bounds. World-space bounds of the level.
CellToWorld	Vector3 CellToWorld(Vector2Int cell)	Input: Vector2Int. Output: Vector3 world position.
WorldToCell	Vector2Int WorldToCell(Vector3 worldPosition)	Input: Vector3. Output: Vector2Int grid coordinate.
ReserveAnchors	IReadOnlyList<Vector2Int> ReserveAnchors(AnchorKind kind, int count)	Inputs: AnchorKind, int. Output: unique read-only reserved cells.
PathExists	bool PathExists(Vector2Int from, Vector2Int to)	Inputs: two Vector2Int values. Output: whether a traversable path exists.
Clear	void Clear()	No value returned. Clears current maze and reservation state.

Inputs / Outputs / Data

- MazeConfig: immutable/read-only generation parameters containing seed, dimensions, difficulty, and braiding ratio as described in the feature plan.
- MazeData: authoritative generated grid containing cells, wall flags, bounds, coordinate conversion information, spawn/exit cells, and data needed by renderer/query consumers.
- AnchorKind: typed category identifying the kind of content reservation being requested.
- Vector2Int, Vector3, Bounds: Unity value types for grid and world-space queries.
- IReadOnlyList<Vector2Int>: returned anchor collection; callers can inspect it but cannot mutate the feature's internal reservation state.

Events — proposed integration contract

- event Action<MazeData> MazeGenerated — raised after successful generation and validation.
- event Action<string> MazeGenerationFailed — raised when generation cannot satisfy the configuration after the documented retry/validation policy; the string contains a reproducible failure identifier including the seed.
- event Action MazeCleared — raised after Clear removes the current maze and reservation state.

Acceptance / Test Contracts

Test	Method	Pass condition
Seed determinism	Generate the same MazeConfig twice.	The two MazeData grids are byte-identical.

Solvability	Generate a maze and inspect SpawnCell and ExitCell.	PathExists(SpawnCell, ExitCell) is true for every supported difficulty.
Anchor safety	Reserve anchors repeatedly from a generated maze.	Every returned anchor is walkable and no cell is returned twice.
Bounds	Query cells inside and outside WorldBounds.	Valid cells convert consistently; out-of-bounds IsWalkable returns false.
Minimum size	Generate the documented minimum 5×5 maze.	Generation succeeds and produces a solvable grid.
Even dimensions	Generate an even-dimension configuration.	No off-by-one exception occurs and the result is solvable.
Stress	Generate a 200×200 maze 100 times.	No allocation spike is observed and no generation exceeds 250 ms.
MVP timing	Generate supported difficulty levels in the normal MVP path.	Each maze generates in under 100 ms and renders correctly.

Internal design relevant to the contract

- MazeGenerator — thin MonoBehaviour façade used to connect the feature to Unity's lifecycle.
- MazeData, Cell, WallFlags, MazeConfig, AnchorRegistry, and MazeRenderer — non-MonoBehaviour feature classes.
- MazeAlgorithm hierarchy — abstract MazeAlgorithm with RecursiveBacktrackerAlgorithm, Prim'sAlgorithm, and BraidedAlgorithm implementations.
- Strategy pattern — the generator selects an interchangeable MazeAlgorithm without branching on a type code.
- Private Class Data — MazeConfig parameters are set once and exposed read-only to protect reproducibility.
- Information Expert — MazeData answers maze queries because it owns the grid. Creator — MazeAlgorithm creates Cell objects because it has the information needed to initialize them.
- Protected Variations — the IMazeQuery boundary protects consumers from changes to the maze implementation.

4. Data Flow Diagrams

The diagrams follow the sample's context-to-zoom structure. The context diagram identifies the feature's external systems and data stores; the zoom diagram decomposes generation into primitive processes. The decision tree makes the control decisions explicit so there are no unexplained 'miracles' in the data flow.

Diagram 0 — Context / Feature Highlight

Diagram 0 — Algernon (maze feature processes highlighted)

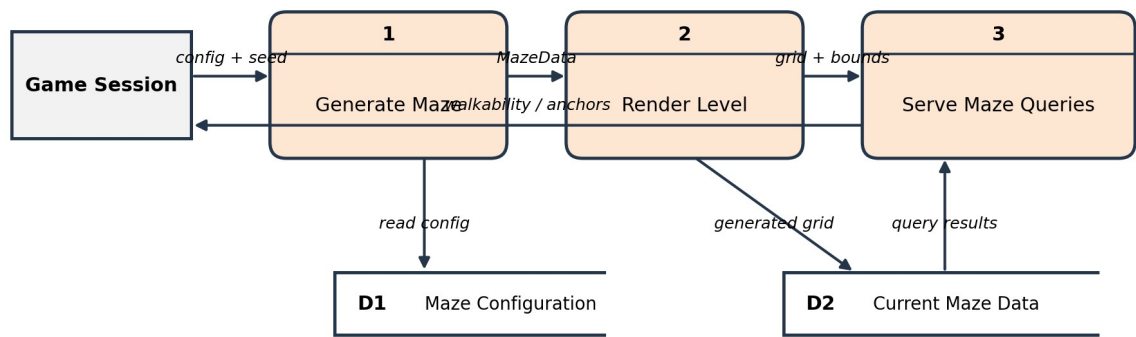
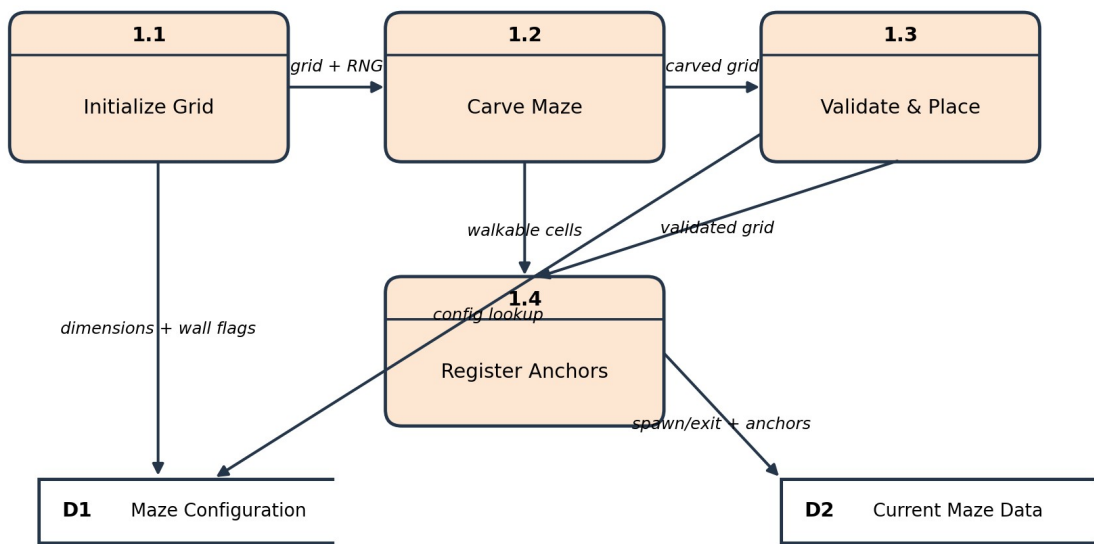


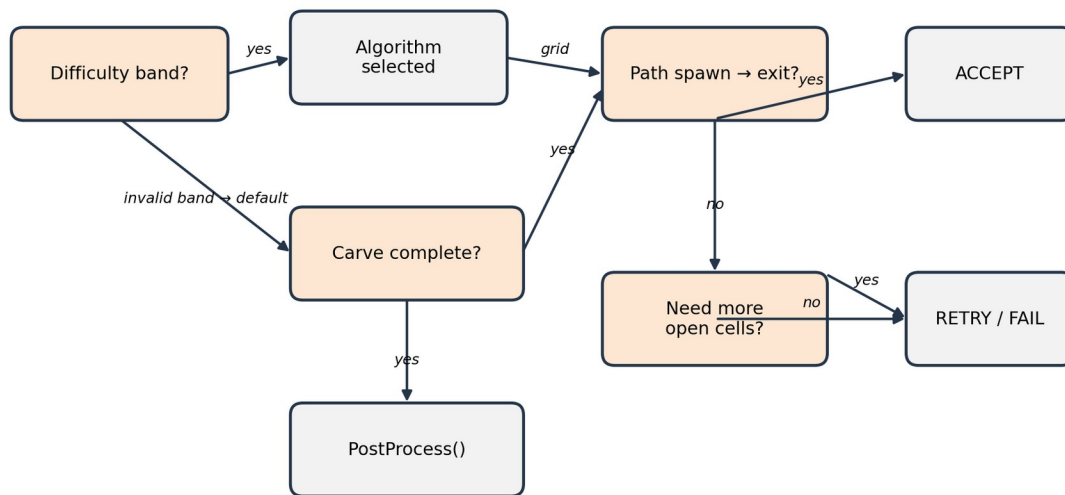
Diagram 1 — Generate Maze Zoom

Diagram 1 — Generate Maze (zoom)



Process Description — 1.2 Carve / Validate

Process Description — 1.2 Carve / Validate Decision Tree



Primitive-process interpretation: Initialize Grid creates the Cell/WallFlags representation; Carve Maze applies the selected MazeAlgorithm using the seeded RNG; Validate & Place verifies usability and places spawn/exit; Register Anchors records cells reserved for downstream features. The data stores are Maze Configuration and Current Maze Data.

5. Timeline

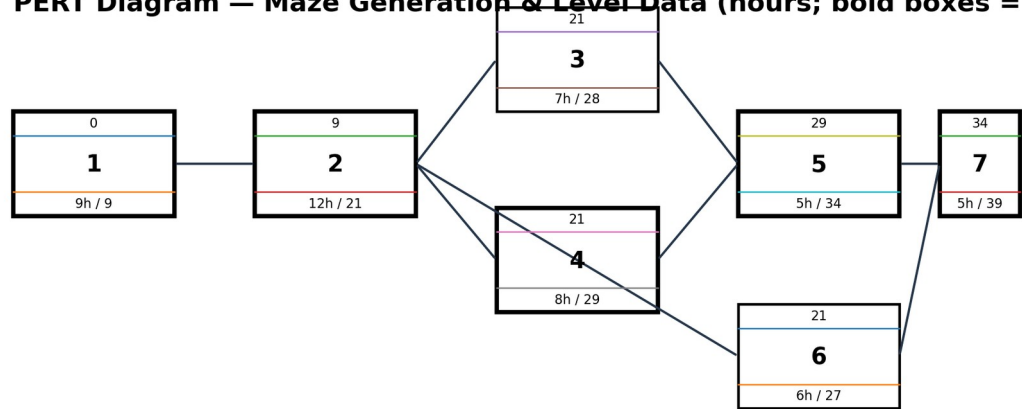
The supplied feature plan gives approximately 52 hours of effort. The work breakdown below preserves that total while making the individual tasks explicit.

Work item	Duration (hours)	Predecessor task(s)
1. Grid/data-model design	9	—
2. Generation algorithm implementations	12	1
3. Seeded determinism + configuration	7	1, 2
4. Spawn/exit placement + solvability validation	5	2, 3
5. Anchor registry + reservation rules	6	1, 4
6. Unity tilemap/wall renderer	5	1, 4
7. Read-only query API + coordinate conversion	3	1, 4, 5
8. Testing, stress testing, bug fixing	3	2, 3, 4, 5, 7
9. Documentation + Champion diagrams	2	1–8

Total planned effort: 52 hours. Critical path for the dependency chain shown in the PERT diagram is 1 → 2 → 4 → 5/7 → 8 → 9; the exact elapsed time depends on which non-critical tasks are run in parallel.

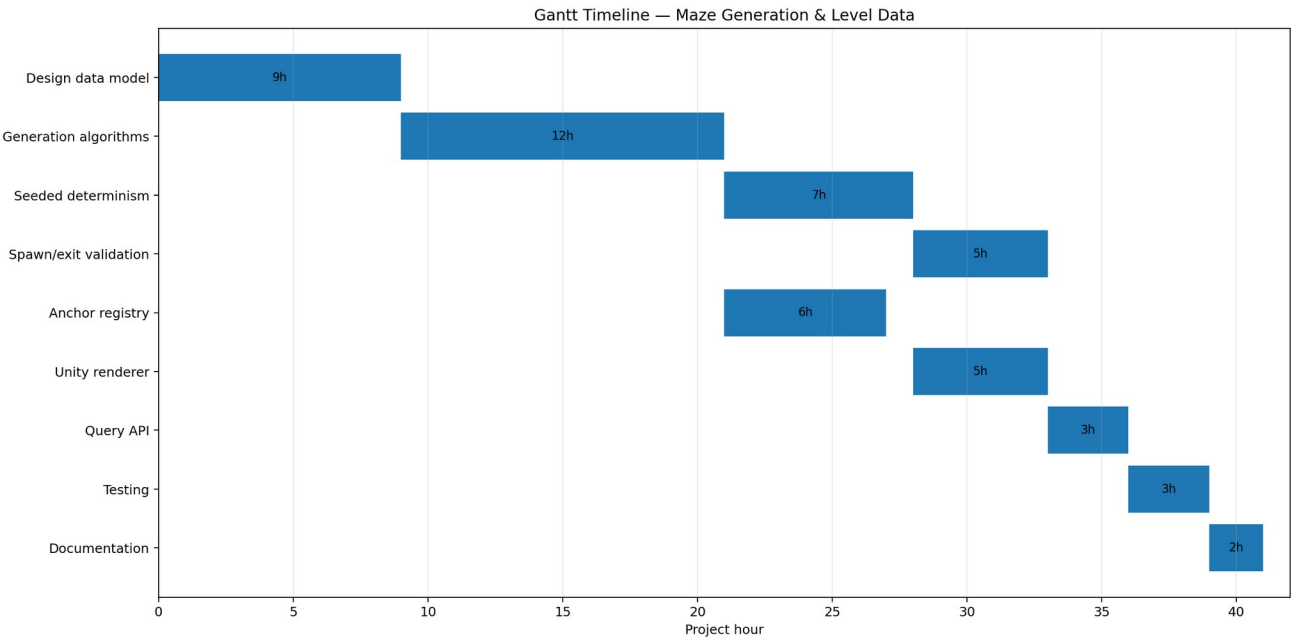
PERT Diagram

PERT Diagram — Maze Generation & Level Data (hours; bold boxes = critical path)



Legend: top row ES | middle task | bottom row duration / EF

Gantt Timeline



Timeline tracking notes

- The first milestone is a headless, solvable MazeData result; rendering follows the grid guarantees.
- Testing is deliberately included because the rest of the game depends on determinism, solvability, and safe anchor guarantees.
- The main technical risk is maintaining deterministic generation while supporting interchangeable algorithms, valid spawn/exit placement, and non-colliding anchor reservations.
- If schedule slips, preserve the MVP guarantees first: solvability, determinism, correct grid queries, and correct rendering.

Source basis

This document uses the supplied Champion Document — Sample as the structural model and the supplied MyFeatureAlgernon feature plan as the feature-specific source. The sample's use-case format, exception

branching, Gane–Sarson DFD structure, decision-tree process description, and PERT/Gantt presentation were carried over and rewritten for F2 — Maze Generation & Level Data.